Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря

Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

ЗВІТ

з Лабораторної роботи №4

«Аналітичні SQL-запити (OLAP)»

з дисципліни

“Бази даних”

Виконали: студенти групи ІМ-55
Горбань А.О

Долгіх Д.М

Перевірив: Хмельницький А.А

Київ - 2026

Тема: Аналітичні SQL-запити (OLAP)

Мета: Освоїти практичні навички маніпулювання даними та складання аналітичних OLAP-запитів у СУБД PostgreSQL (pgAdmin). Навчитися застосовувати агрегатні функції (COUNT, SUM, AVG, MIN, MAX), оператори групування (GROUP BY), фільтрацію згрупованих результатів (HAVING), різноманітні типи з'єднань таблиць (INNER JOIN, LEFT JOIN, CROSS JOIN) та вкладені підзапити (Subqueries).

2. КОД

```
-- Запит 1
SELECT MIN(order_date) AS first_order_date, MAX(order_date) AS last_order_date
FROM orders;

-- Запит 2
SELECT COUNT(*) AS total_orders
FROM orders;

-- Запит 3
SELECT payment_method, SUM(price) AS revenue
FROM payment
GROUP BY payment_method;

-- Запит 4
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent
FROM orders
GROUP BY client_id;

-- Запит 5
SELECT c.client_id, c.client_name, SUM(o.order_price) AS total_spent
FROM client c
LEFT JOIN orders o ON c.client_id = o.client_id
GROUP BY c.client_id, c.client_name
HAVING SUM(o.order_price) > 5000;

-- Запит 6
SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price
FROM product p
JOIN category c ON p.category_id = c.category_id
GROUP BY c.category_name;

-- Запит 7
SELECT o.order_date, o.order_price, c.client_name || ' ' || c.last_name AS full_name, c.email,
c.phone_number, p.product_name
FROM orders o
INNER JOIN client c ON c.client_id = o.client_id
INNER JOIN order_item oi ON oi.order_id = o.order_id
INNER JOIN product p ON p.product_id = oi.product_id;

-- Запит 8
SELECT p.product_name, p.price
FROM product p
LEFT JOIN order_item oi ON p.product_id = oi.product_id
WHERE oi.order_item_id IS NULL;
```

```
-- Запит 9
SELECT w.last_name AS worker_name, c.client_name
FROM worker w
CROSS JOIN client c;

-- Запит 10
SELECT client_name, last_name,
  (SELECT COUNT(*) FROM orders WHERE orders.client_id = client.client_id) AS
orders_count
FROM client;

-- Запит 11
SELECT product_name, price
FROM product
WHERE price > (SELECT AVG(price) FROM product);

-- Запит 12
SELECT c.client_name, SUM(o.order_price) AS total_spent
FROM client c
JOIN orders o ON c.client_id = o.client_id
GROUP BY c.client_id, c.client_name
HAVING SUM(o.order_price) > (SELECT AVG(order_price) FROM orders);
```

3. ХІД ВИКОНАННЯ РОБОТИ ТА ОПИС SQL-ЗАПИТІВ

Частина 1. Агрегаційні функції, групування (GROUP BY) та фільтрація (HAVING)

Запит 1. Дата першого та останнього замовлення

Опис: Визначає найранішню та найновішу дати реєстрації замовлень у системі за допомогою агрегатних функцій MIN та MAX.

```
SELECT MIN(order_date) AS first_order_date, MAX(order_date) AS last_order_date FROM
orders;
```

Запит 2. Загальна кількість замовлень

Опис: Обчислює загальну кількість усіх оформлених замовлень у базі даних інтернет-магазину за допомогою функції COUNT.

```
SELECT COUNT(*) AS total_orders
FROM orders;
```

Запит 3. Загальна виручка за кожним методом оплати

Опис: Підраховує сумарний дохід від здійснених платежів (SUM), згрупований за способом проведення транзакцій (GROUP BY).

```
SELECT payment_method, SUM(price) AS revenue
FROM payment
GROUP BY payment_method;
```

Запит 4. Середня сума витрачених коштів на замовлення для клієнта

Опис: Обчислює середній чек покупок (AVG) окремо для кожного клієнта з округленням до двох знаків після коми.

```
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent
FROM orders
GROUP BY client_id;
```

Запит 5. Клієнти, які витратили понад 5000 грн

Опис: Об'єднує таблиці клієнтів та замовлень, підраховує сумарні витрати кожного покупця та за допомогою HAVING відбирає лише VIP-клієнтів із сумою покупок понад 5000 грн.

```
SELECT c.client_id, c.client_name, SUM(o.order_price) AS total_spent
FROM client c
LEFT JOIN orders o ON c.client_id = o.client_id
GROUP BY c.client_id, c.client_name
HAVING SUM(o.order_price) > 5000;
```

Запит 6. Середня ціна продуктів у кожній категорії
Опис: Виконує багатотабличну агрегацію, об'єднуючи товари з категоріями та розраховуючи середню вартість одиниці товару в кожній категорії.

```
SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price
FROM product p
JOIN category c ON p.category_id = c.category_id
GROUP BY c.category_name;
```

Частина 2. Оператори об'єднання таблиць (JOIN)
Запит 7. Детальна інформація про замовлення, клієнтів та товари (INNER JOIN)
Опис: Об'єднує дані з чотирьох таблиць (orders, client, order_item, product) для формування повної накладної замовлення з датою, сумою, ПІБ, контактами покупця та назвою товару.

```
SELECT o.order_date, o.order_price, c.client_name || ' ' || c.last_name AS full_name, c.email,
c.phone_number, p.product_name
FROM orders o
INNER JOIN client c ON c.client_id = o.client_id
INNER JOIN order_item oi ON oi.order_id = o.order_id INNER JOIN product p ON
p.product_id = oi.product_id;
```

Запит 8. Товари, які ще жодного разу не замовляли (LEFT JOIN) Опис: За допомогою лівого зовнішнього з'єднання (LEFT JOIN) та перевірки на NULL виявляє позиції з каталогу товарів, які ще не фігурували в жодному замовленні.

```
SELECT p.product_name, p.price
FROM product p
LEFT JOIN order_item oi ON p.product_id = oi.product_id
WHERE oi.order_item_id IS NULL;
```

Запит 9. Усі можливі комбінації працівників і клієнтів (CROSS JOIN) Опис: Створює декартовий добуток між таблицями працівників та клієнтів, генеруючи всі можливі пари «працівник — клієнт».

```
SELECT w.last_name AS worker_name, c.client_name
FROM worker w
CROSS JOIN client c;
```

Частина 3. Запити з використанням підзапитів
Запит 10. Кількість замовлень кожного клієнта (підзапит у SELECT) Опис: Для кожного зареєстрованого клієнта динамічно обчислює загальну кількість зроблених ним замовлень безпосередньо у секції вибірки стовпців SELECT.

```
SELECT client_name, last_name,
(SELECT COUNT(*) FROM orders WHERE orders.client_id = client.client_id) AS
orders_count
FROM client;
```

Запит 11. Товари, дорожчі за середню ціну по каталогу (підзапит у WHERE) Опис:

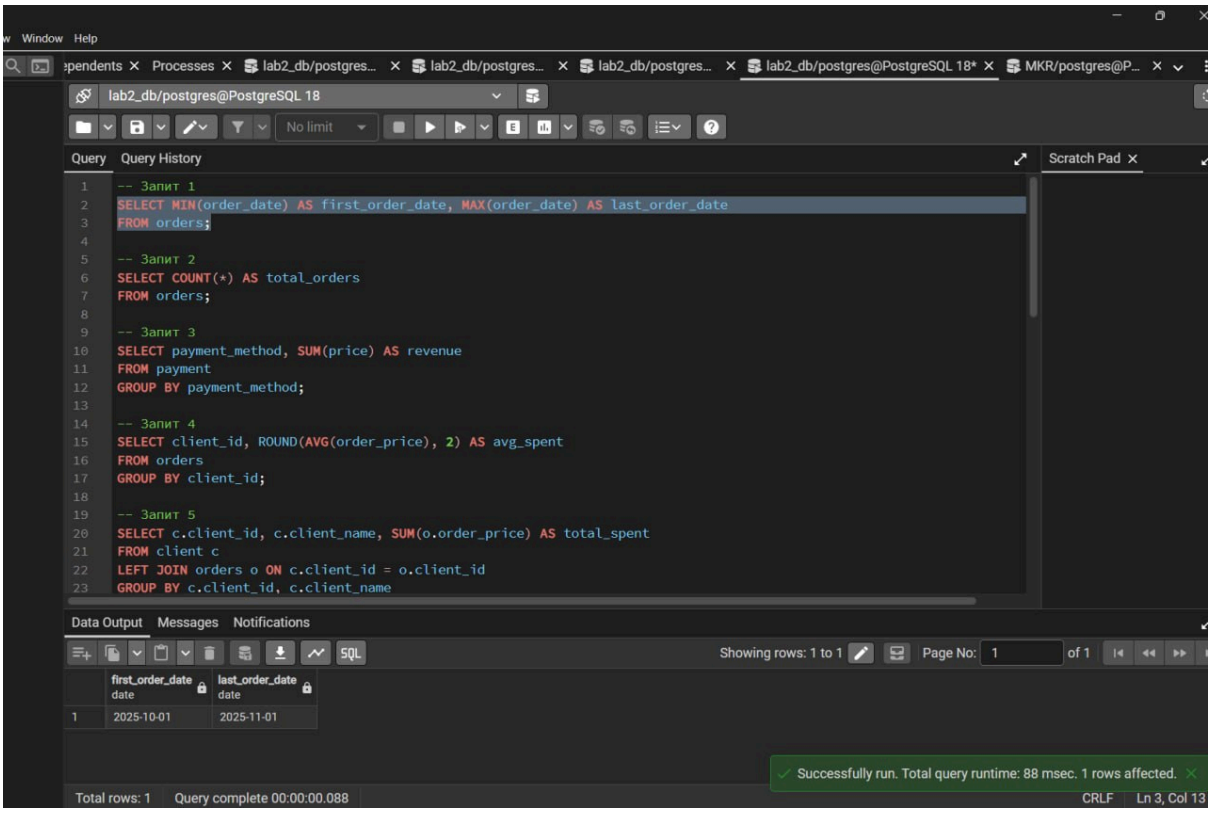
Відбирає товари преміум-сегмента, вартість яких перевищує середню ціну всіх товарів у магазині, обчислену вкладеним підзапитом у WHERE.

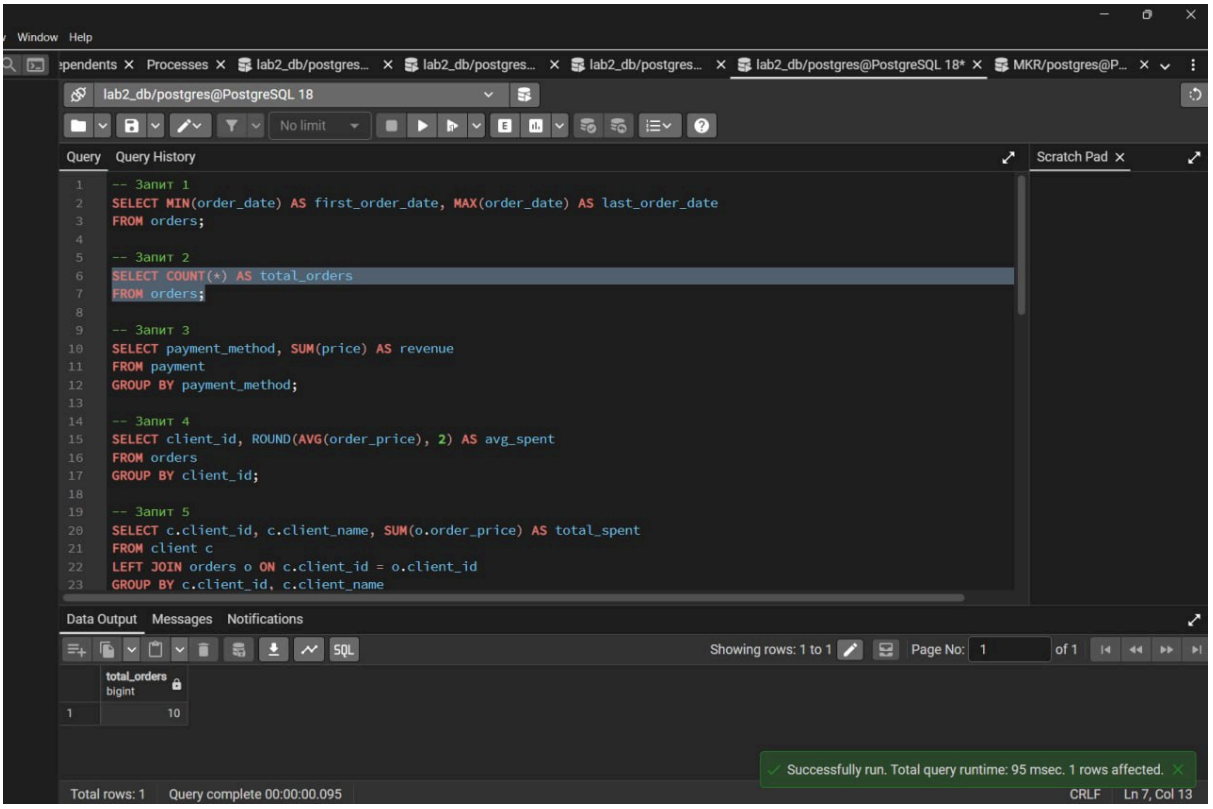
```
SELECT product_name, price
FROM product
WHERE price > (SELECT AVG(price) FROM product);
```

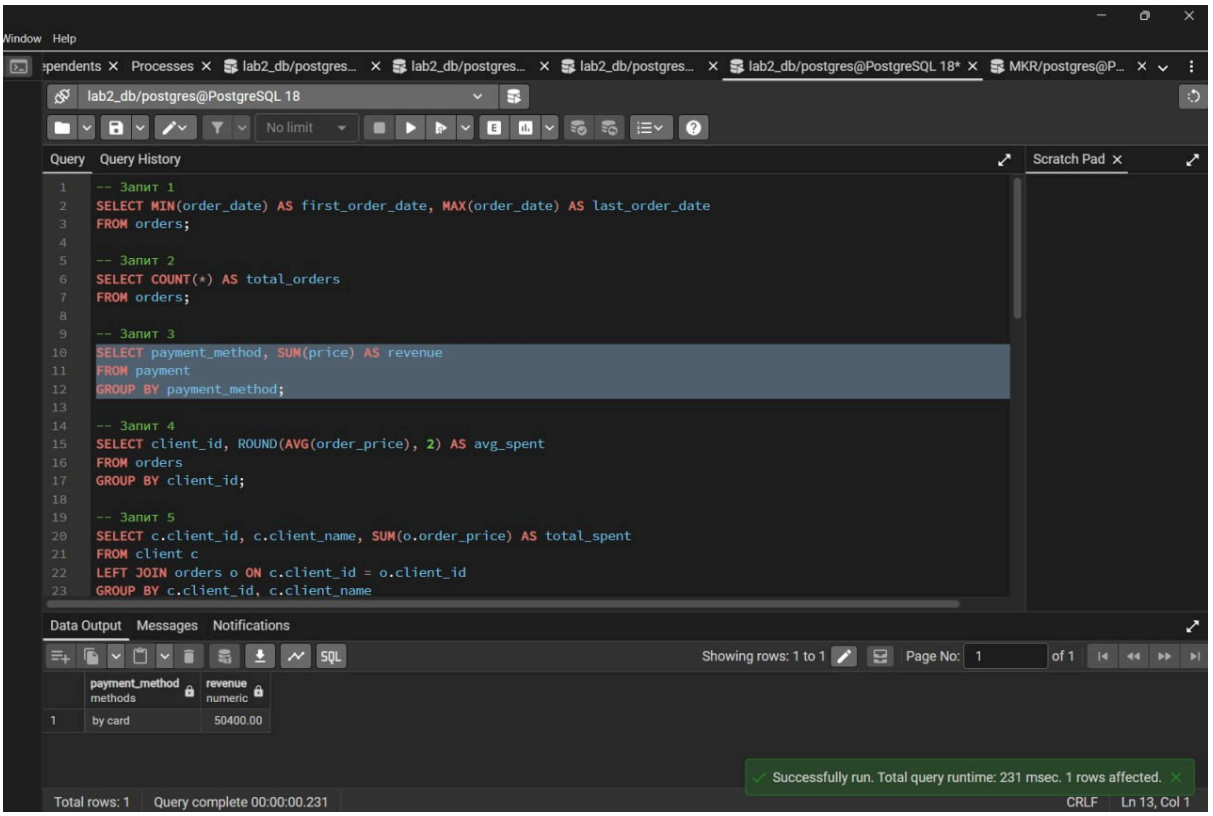
Запит 12. Клієнти із сумарними витратами вище середнього чека (підзапит у HAVING)
Опис: Групує витрати клієнтів за замовленнями та за допомогою підзапиту в HAVING залишає лише тих покупців, чия загальна сума покупок перевищує середню вартість одного замовлення по всій базі.

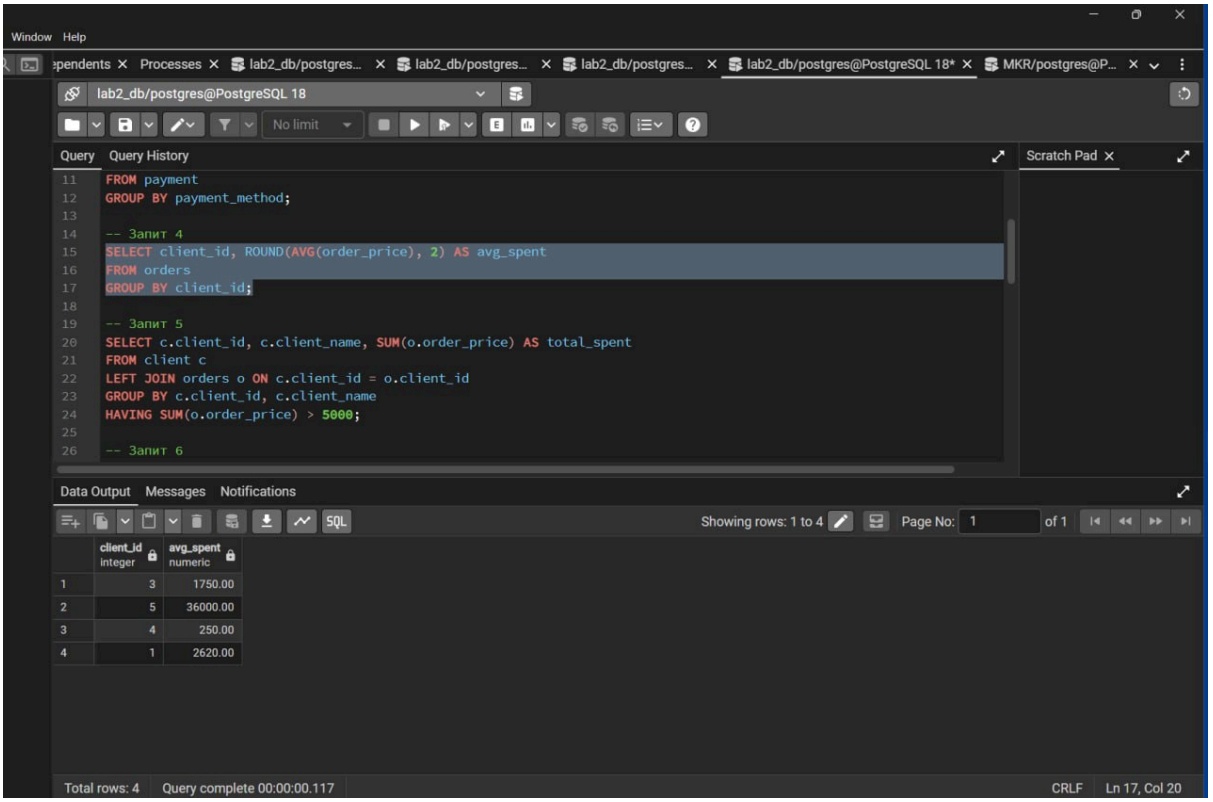
```
SELECT c.client_name, SUM(o.order_price) AS total_spent
FROM client c
JOIN orders o ON c.client_id = o.client_id
GROUP BY c.client_id, c.client_name
HAVING SUM(o.order_price) > (SELECT AVG(order_price) FROM orders);
```

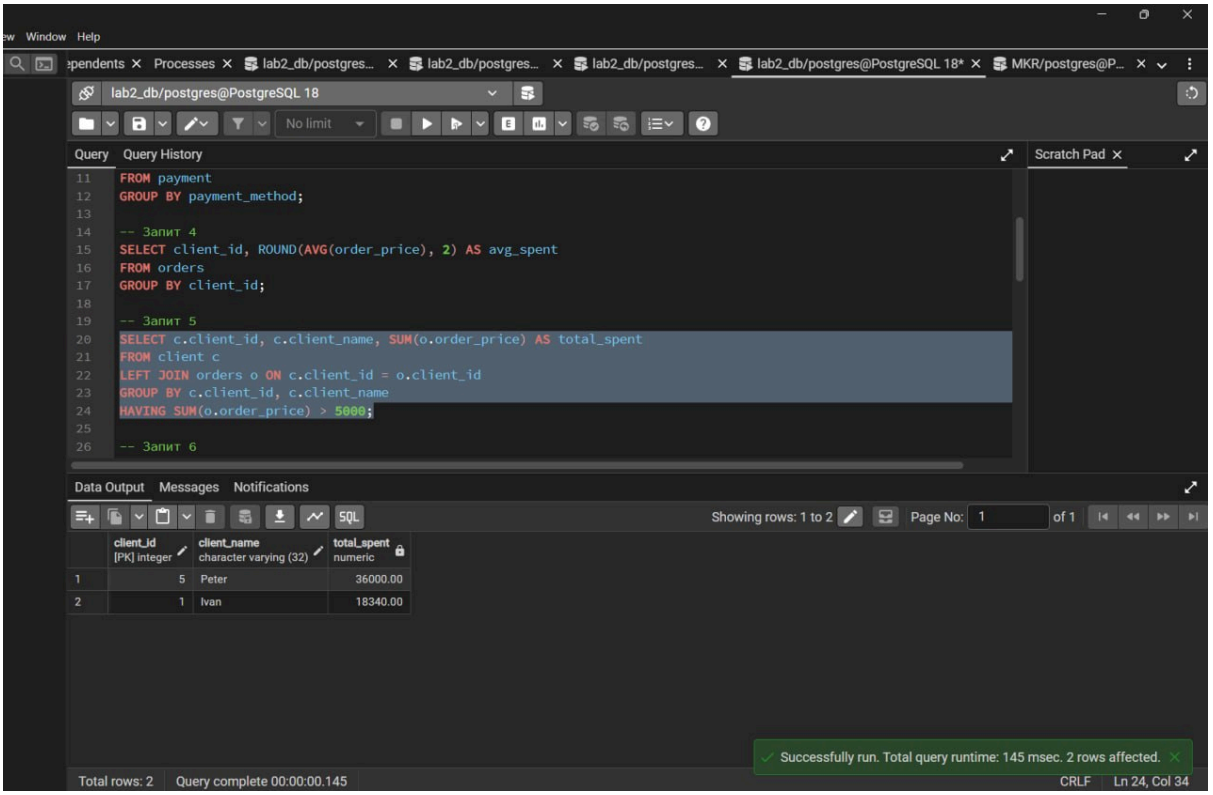
Результати виконання

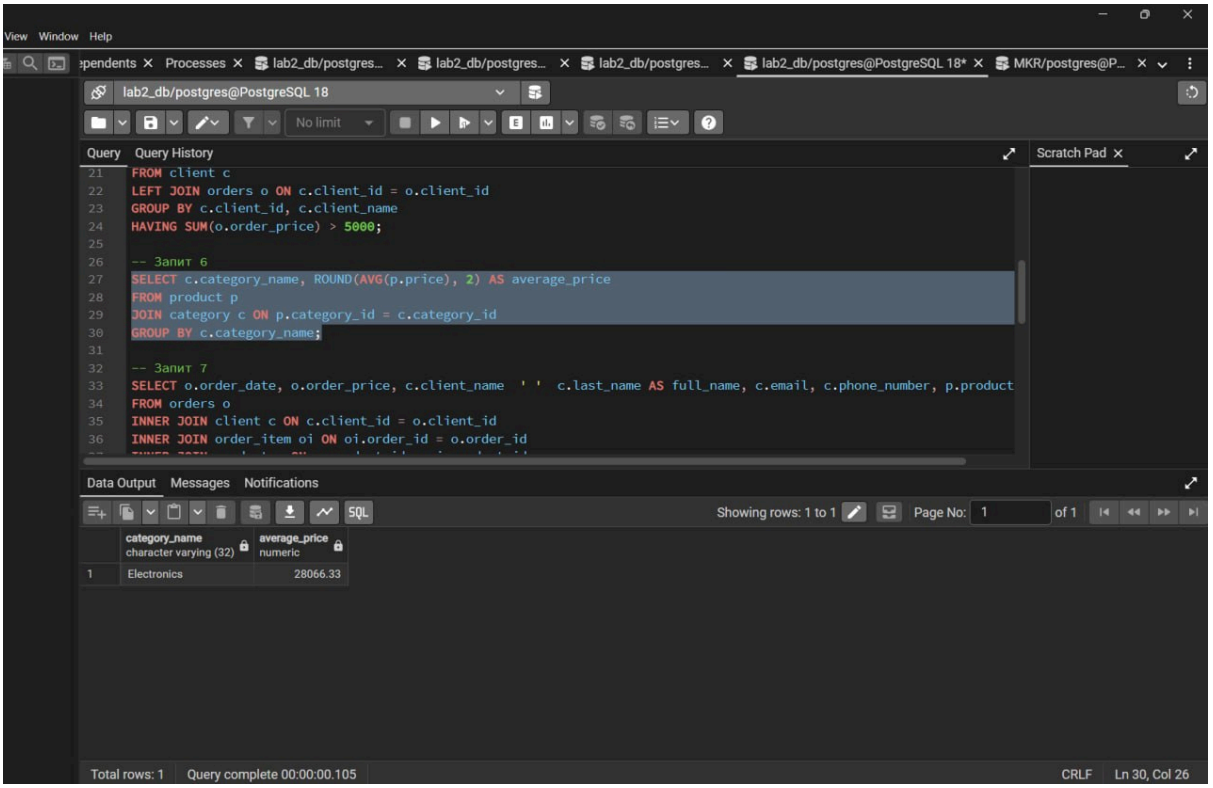


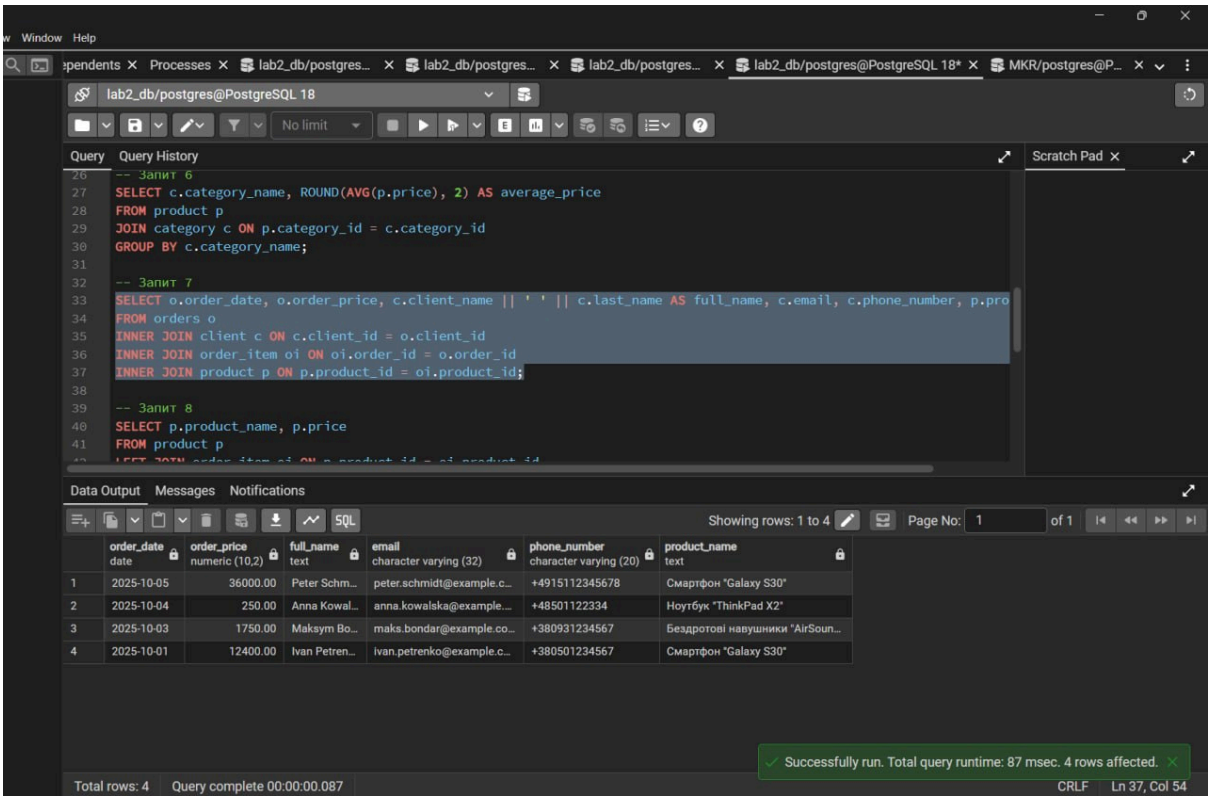

Window Help
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
-- Запит 1
SELECT MIN(order_date) AS first_order_date, MAX(order_date) AS last_order_date
FROM orders;
-- Запит 2
SELECT COUNT(*) AS total_orders
FROM orders;
-- Запит 3
SELECT payment_method, SUM(price) AS revenue
FROM payment
GROUP BY payment_method;
-- Запит 4
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent
FROM orders
GROUP BY client_id;
-- Запит 5
SELECT c.client_id, c.client_name, SUM(o.order_price) AS total_spent
FROM client c
LEFT JOIN orders o ON c.client_id = o.client_id
GROUP BY c.client_id, c.client_name
Data Output Messages Notifications
Showing rows: 1 to 1 Page No: 1 of 1
total_orders bigint
1 10
Successfully run. Total query runtime: 95 msec. 1 rows affected.
Total rows: 1 Query complete 00:00:00.095
CRLF Ln 7, Col 13

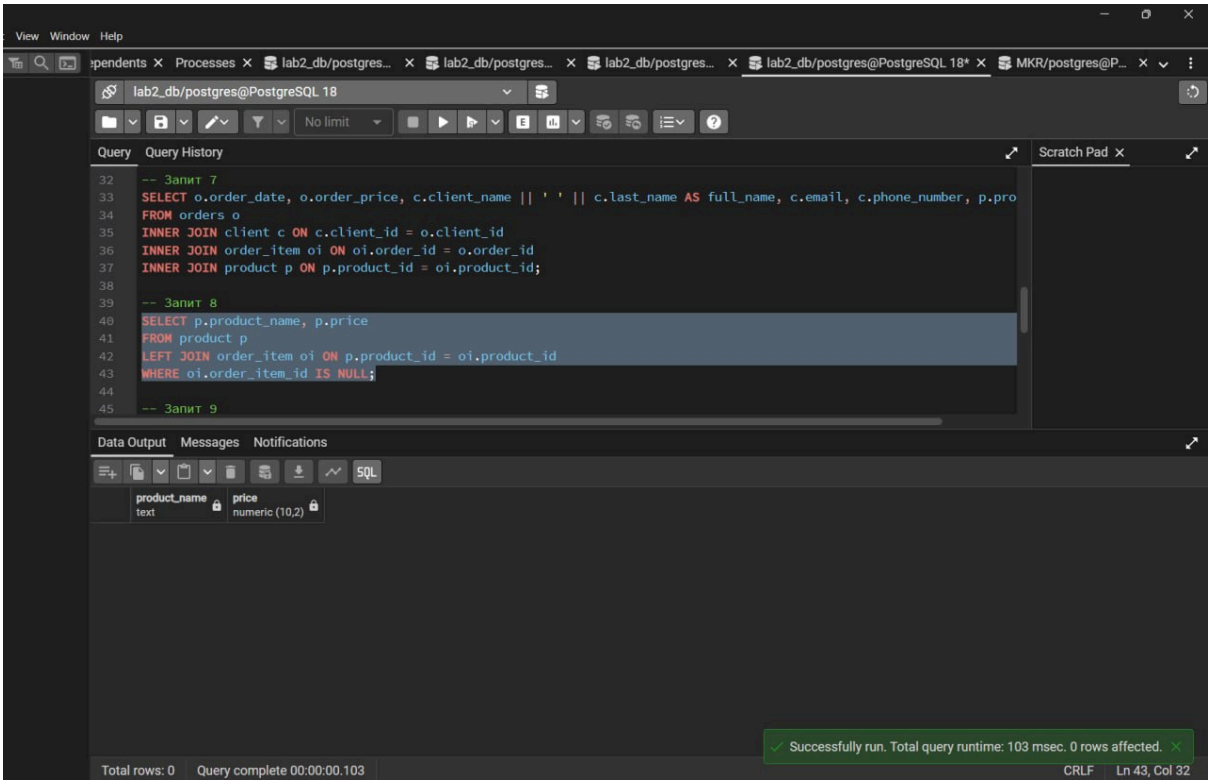

Window Help
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
-- Запит 1
SELECT MIN(order_date) AS first_order_date, MAX(order_date) AS last_order_date
FROM orders;
-- Запит 2
SELECT COUNT(*) AS total_orders
FROM orders;
-- Запит 3
SELECT payment_method, SUM(price) AS revenue
FROM payment
GROUP BY payment_method;
-- Запит 4
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent
FROM orders
GROUP BY client_id;
-- Запит 5
SELECT c.client_id, c.client_name, SUM(o.order_price) AS total_spent
FROM client c
LEFT JOIN orders o ON c.client_id = o.client_id
GROUP BY c.client_id, c.client_name
Data Output Messages Notifications
Showing rows: 1 to 1 Page No: 1 of 1
payment_method methods
revenue numeric
1 by card 50400.00
Successfully run. Total query runtime: 231 msec. 1 rows affected.
Total rows: 1 Query complete 00:00:00.231
CRLF Ln 13, Col 1

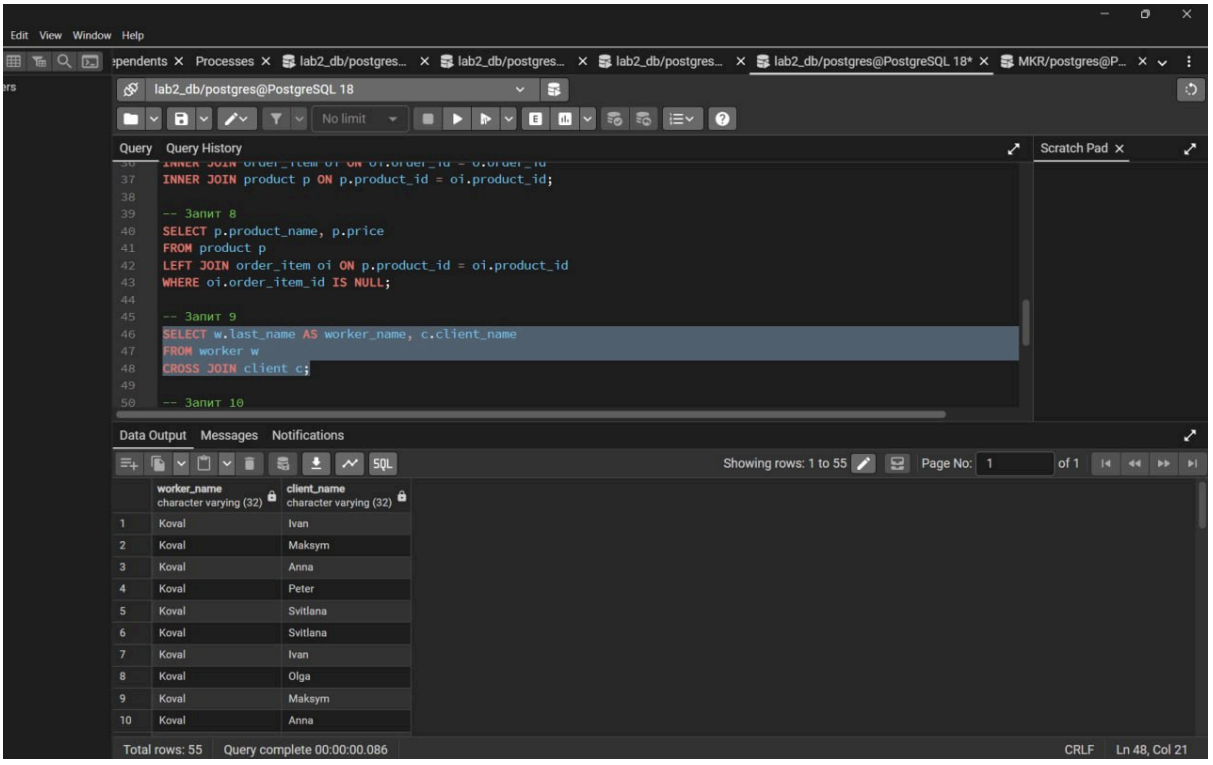

Window Help
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
FROM payment
GROUP BY payment_method;
-- Запит 4
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent
FROM orders
GROUP BY client_id;
-- Запит 5
SELECT c.client_id, c.client_name, SUM(o.order_price) AS total_spent
FROM client c
LEFT JOIN orders o ON c.client_id = o.client_id
GROUP BY c.client_id, c.client_name
HAVING SUM(o.order_price) > 5000;
-- Запит 6
Data Output Messages Notifications
Showing rows: 1 to 4
Page No: 1 of 1
client_id integer
avg_spent numeric
1 3 1750.00
2 5 36000.00
3 4 250.00
4 1 2620.00
Total rows: 4
Query complete 00:00:00.117
CRLF
Ln 17, Col 20

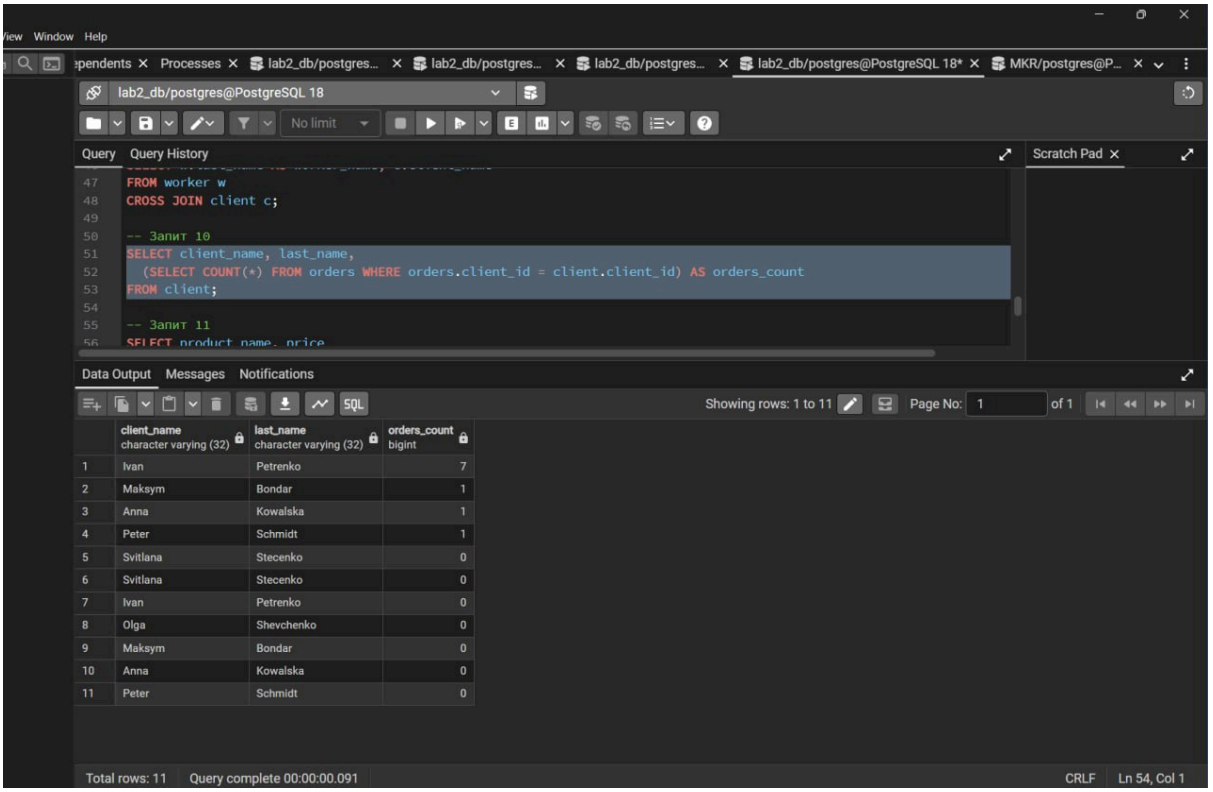

Window Help
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
FROM payment
GROUP BY payment_method;
-- Запит 4
SELECT client_id, ROUND(AVG(order_price), 2) AS avg_spent
FROM orders
GROUP BY client_id;
-- Запит 5
SELECT c.client_id, c.client_name, SUM(o.order_price) AS total_spent
FROM client c
LEFT JOIN orders o ON c.client_id = o.client_id
GROUP BY c.client_id, c.client_name
HAVING SUM(o.order_price) > 5000;
-- Запит 6
Data Output Messages Notifications
Showing rows: 1 to 2
Page No: 1 of 1
client_id [PK] integer
client_name character varying (32)
total_spent numeric
1 5 Peter 36000.00
2 1 Ivan 18340.00
Successfully run. Total query runtime: 145 msec. 2 rows affected.
Total rows: 2
Query complete 00:00:00.145
CRLF
Ln 24, Col 34


lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
-- Запит 6
SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price
FROM product p
JOIN category c ON p.category_id = c.category_id
GROUP BY c.category_name;
-- Запит 7
SELECT o.order_date, o.order_price, c.client_name ' ' c.last_name AS full_name, c.email, c.phone_number, p.product
FROM orders o
INNER JOIN client c ON c.client_id = o.client_id
INNER JOIN order_item oi ON oi.order_id = o.order_id
Data Output
Messages
Notifications
Showing rows: 1 to 1
Page No: 1 of 1
category_name character varying (32)
average_price numeric
1 Electronics 28066.33
Total rows: 1
Query complete 00:00:00.105
CRLF
Ln 30, Col 26


lab2_db/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
-- Запит 6
SELECT c.category_name, ROUND(AVG(p.price), 2) AS average_price
FROM product p
JOIN category c ON p.category_id = c.category_id
GROUP BY c.category_name;
-- Запит 7
SELECT o.order_date, o.order_price, c.client_name || ' ' || c.last_name AS full_name, c.email, c.phone_number, p.pro
FROM orders o
INNER JOIN client c ON c.client_id = o.client_id
INNER JOIN order_item oi ON oi.order_id = o.order_id
INNER JOIN product p ON p.product_id = oi.product_id;
-- Запит 8
SELECT p.product_name, p.price
FROM product p
Data Output
Messages
Notifications
Showing rows: 1 to 4
Page No: 1 of 1
order_date date
order_price numeric (10,2)
full_name text
email character varying (32)
phone_number character varying (20)
product_name text
1 2025-10-05 36000.00 Peter Schm... peter.schmidt@example.c... +4915112345678 Смартфон "Galaxy S30"
2 2025-10-04 250.00 Anna Kowal... anna.kowalska@example.... +48501122334 Ноутбук "ThinkPad X2"
3 2025-10-03 1750.00 Maksym Bo... maks.bondar@example.co... +380931234567 Бездротові навушники "AirSoun...
4 2025-10-01 12400.00 Ivan Petren... ivan.petrenko@example.c... +380501234567 Смартфон "Galaxy S30"
Successfully run. Total query runtime: 87 msec. 4 rows affected.
Total rows: 4
Query complete 00:00:00.087
CRLF
Ln 37, Col 54


View Window Help
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
-- Запит 7
SELECT o.order_date, o.order_price, c.client_name || ' ' || c.last_name AS full_name, c.email, c.phone_number, p.pro
FROM orders o
INNER JOIN client c ON c.client_id = o.client_id
INNER JOIN order_item oi ON oi.order_id = o.order_id
INNER JOIN product p ON p.product_id = oi.product_id;
-- Запит 8
SELECT p.product_name, p.price
FROM product p
LEFT JOIN order_item oi ON p.product_id = oi.product_id
WHERE oi.order_item_id IS NULL;
-- Запит 9
Data Output Messages Notifications
product_name text
price numeric (10,2)
Successfully run. Total query runtime: 103 msec. 0 rows affected.
Total rows: 0 Query complete 00:00:00.103
CRLF Ln 43, Col 32


Edit View Window Help
lab2_db/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
INNER JOIN product p ON p.product_id = oi.product_id;
-- Запит 8
SELECT p.product_name, p.price
FROM product p
LEFT JOIN order_item oi ON p.product_id = oi.product_id
WHERE oi.order_item_id IS NULL;
-- Запит 9
SELECT w.last_name AS worker_name, c.client_name
FROM worker w
CROSS JOIN client c;
-- Запит 10
Data Output Messages Notifications
Showing rows: 1 to 55
Page No: 1 of 1
worker_name character varying (32)
client_name character varying (32)
1 Koval Ivan
2 Koval Maksym
3 Koval Anna
4 Koval Peter
5 Koval Svitlana
6 Koval Svitlana
7 Koval Ivan
8 Koval Olga
9 Koval Maksym
10 Koval Anna
Total rows: 55 Query complete 00:00:00.086
CRLF Ln 48, Col 21

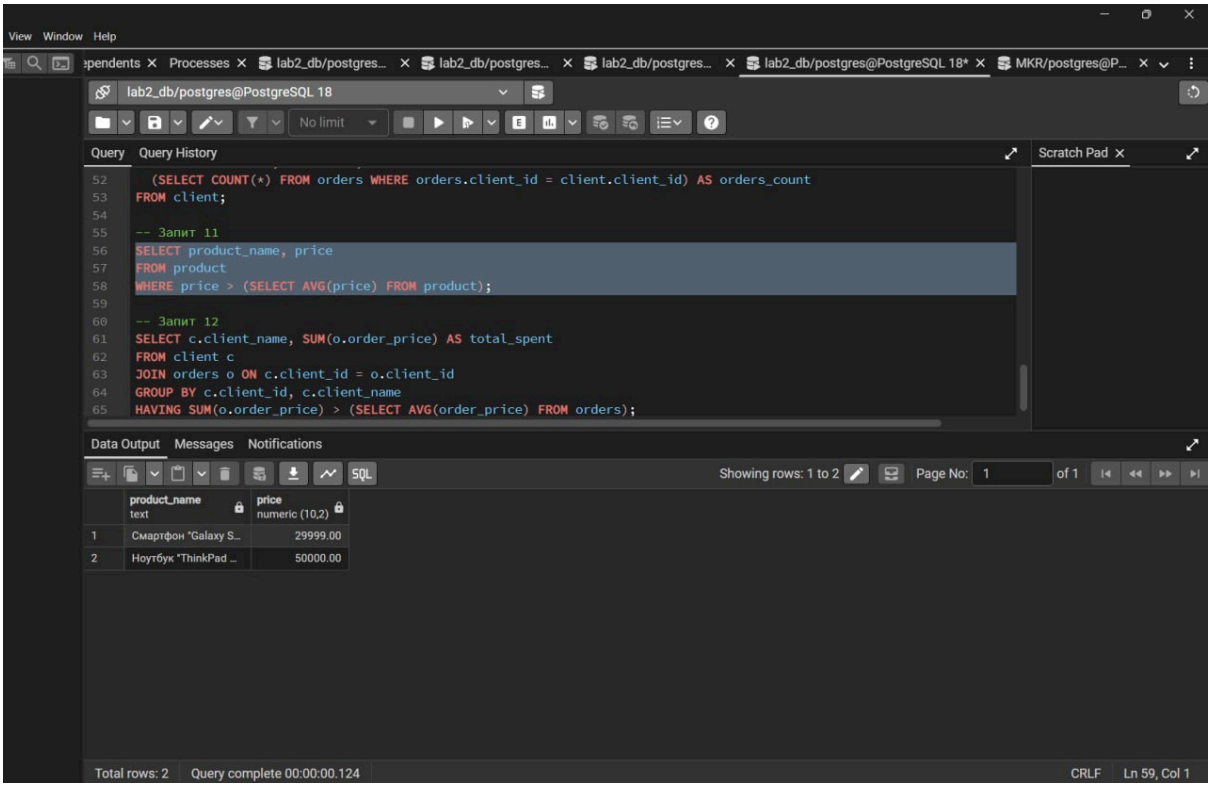


```
FROM worker w
CROSS JOIN client c;

-- Запит 10
SELECT client_name, last_name,
  (SELECT COUNT(*) FROM orders WHERE orders.client_id = client.client_id) AS orders_count
FROM client;

-- Запит 11
SELECT product_name, price
```

	client_name character varying (32)	last_name character varying (32)	orders_count bigint
1	Ivan	Petrenko	7
2	Maksym	Bondar	1
3	Anna	Kowalska	1
4	Peter	Schmidt	1
5	Svitlana	Stecenko	0
6	Svitlana	Stecenko	0
7	Ivan	Petrenko	0
8	Olga	Shevchenko	0
9	Maksym	Bondar	0
10	Anna	Kowalska	0
11	Peter	Schmidt	0

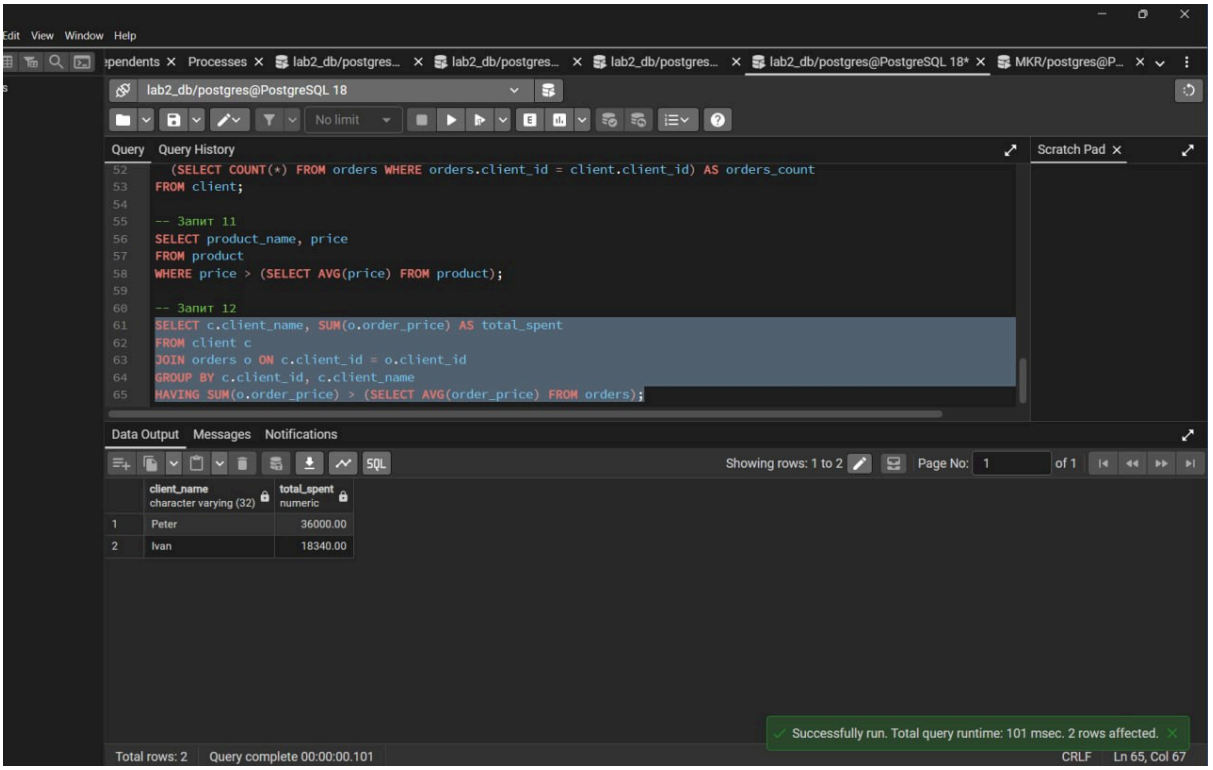


```
  (SELECT COUNT(*) FROM orders WHERE orders.client_id = client.client_id) AS orders_count
FROM client;

-- Запит 11
SELECT product_name, price
FROM product
WHERE price > (SELECT AVG(price) FROM product);

-- Запит 12
SELECT c.client_name, SUM(o.order_price) AS total_spent
FROM client c
JOIN orders o ON c.client_id = o.client_id
GROUP BY c.client_id, c.client_name
HAVING SUM(o.order_price) > (SELECT AVG(order_price) FROM orders);
```

	product_name text	price numeric (10,2)
1	Смартфон "Galaxy S...	29999.00
2	Ноутбук "ThinkPad ...	50000.00



Висновок:

Під час виконання лабораторної роботи №4 було опановано принципи побудови аналітичних SQL-запитів (OLAP) у середовищі СУБД PostgreSQL (pgAdmin) на основі розробленої схеми бази даних інтернет-магазину.

У ході роботи було закріплено практичні навички використання агрегатних функцій (COUNT, SUM, AVG, MIN, MAX) для обчислення зведеної статистики, групування записів за допомогою оператора GROUP BY та фільтрації згрупованих результатів за агрегованими умовами з використанням речення HAVING. Було реалізовано багатотабличні запити із застосуванням різних типів об'єднань (INNER JOIN, LEFT JOIN, CROSS JOIN), що дозволило коректно поєднувати дані з нормалізованих таблиць клієнтів, замовлень, товарів, категорій, працівників та платежів. Також було відпрацьовано написання вкладених і корельованих підзапитів у секціях SELECT, WHERE та HAVING для виконання багаторівневого аналізу даних.

Усі 12 розроблених запитів успішно виконано без помилок, а отримані результати підтвердили їхню практичну цінність для формування управлінської звітності інтернет-магазину: оцінки загальної виручки за методами оплати, розрахунку середнього чека, визначення VIP-клієнтів, аналізу цінової політики категорій та виявлення товарів, які не користуються попитом.